

Assignment 2: Email Intent Extraction System

Prompt Engineering for Structured Information Extraction

Student ID: 12349031, AKINOLA, Lanre Olusegun

Course: 700.390 Advanced Topics in Neurocomputing:
Explainability, Neuro-Symbolic Computing, AI Agents, LLM,
Quantum Computing, PINN

1 Introduction

This assignment builds an LLM-based system to extract structured information from student emails. The goal is to design a prompt that reliably reads an incoming email and outputs a valid JSON object with predefined fields. The prompt needs to handle different scenarios: clear requests, missing information, ambiguous intent, and emails with irrelevant content.

The core idea is to treat the prompt as an engineering artifact. Instead of writing a one-time message, the prompt is designed as a contract that specifies what the model should do, what constraints it must follow, and exactly what format the output should have.

2 System Design

2.1 Model

The model used for this assignment is `Qwen/Qwen2.5-1.5B-Instruct` [1]. It has 1.5 billion parameters and is instruction-tuned, which means it is designed to follow explicit instructions. It was run on Google Colab using an A100 GPU. This model is larger and better suited for tasks that require following structured output formats.

2.2 Prompt Contract

The prompt was designed as a system message with five parts:

- **Role:** Academic administration assistant
- **Task:** Extract student data and classify the intent of the email
- **Context:** Use only the email content, do not rely on outside knowledge
- **Constraints:** Do not invent or guess any missing information
- **Output:** Return valid JSON only, with no extra text

The required output format includes six fields: `student_name`, `student_id`, `email_intent`, `course_or_exam`, `confidence`, and `missing_information`.

The full system prompt is shown below:

Listing 1: System prompt used for email intent extraction

```
You are an academic administration assistant.
Your task is to read a student email and extract information from it.
```

```

Extract exactly these fields:
- student_name: full name of the student, or null if not given
- student_id: student ID number as a string, or null if not given
- email_intent: one of "register", "deregister", or "unknown" if unclear
- course_or_exam: the course or exam mentioned, or null if not given
- confidence: "high" if intent is clear, "medium" if somewhat clear,
  "low" if ambiguous
- missing_information: list of field names that are missing,
  e.g. ["student_id"]

Rules:
- Use ONLY what is written in the email. Do not guess or invent anything.
- If intent is ambiguous or unclear, set email_intent to "unknown"
  and confidence to "low".
- Return ONLY valid JSON. No explanation, no markdown, no extra text.

Output format:
{"student_name": "...", "student_id": "...", "email_intent": "...",
 "course_or_exam": "...", "confidence": "...", "missing_information": []}

```

The temperature was set to 0.1 to get consistent, deterministic outputs. A low temperature is appropriate here because the task is about extraction, not generation.

3 Test Emails

Five test emails were written to cover the required scenarios. Table 1 describes each one.

Table 1: Test email scenarios

Email	Scenario	Description
1	Clear registration	Student provides full name, student ID, and course name. Intent is explicitly to register.
2	Clear deregistration	Student states they want to deregister from an exam. All fields present.
3	Missing student ID	Student wants to register but does not mention their student ID.
4	Ambiguous intent	Student says they registered before but is unsure whether to stay or withdraw.
5	Extra irrelevant info	Email includes unrelated sentences about weather and hiking, but a clear registration request is present.

4 Results

All five emails produced valid JSON output. Table 2 shows the extracted fields for each email. The model returned output in the correct format in all cases, which shows the prompt structure worked well for enforcing the output format.

Table 2: Extraction results for all five test emails

Email	Intent	Confidence	ID found	Missing field	Valid JSON
1	register	high	yes	none	
2	deregister	high	yes	none	
3	register	high	null	<i>not reported</i>	*
4	register	high	yes	none	*
5	register	high	yes	none	

* Valid JSON but contains incorrect field values (see Discussion)

Average generation time per email was 1.99 seconds. Table 3 shows the time for each email.

Table 3: Generation time per email

Email	Scenario	Time (s)
1	Clear registration	3.12
2	Clear deregistration	1.86
3	Missing student ID	1.50
4	Ambiguous intent	1.76
5	Extra irrelevant info	1.64
Average		1.98

4.1 Individual Outputs

Emails 1 and 2 were fully correct. The model correctly identified the intent, extracted all fields, and returned them in the required format.

For email 3 (missing student ID), the model correctly set `student_id` to `null`, but the `missing_information` field was left as an empty list instead of `["student_id"]`. The intent and name were extracted correctly.

For email 4 (ambiguous intent), the model returned `email_intent: "register"` with `confidence: "high"`, even though the student explicitly stated they were unsure whether to stay or withdraw. The expected output would be `email_intent: "unknown"` and `confidence: "low"`.

For email 5, the model correctly ignored the unrelated sentences about the weather and hiking and extracted the registration request with the correct student details.

5 Discussion

The system worked well for the straightforward cases. The model followed the prompt format consistently and always returned valid JSON, which means the output constraint in the prompt was effective.

The two failure cases reveal limitations of small instruction-tuned models:

Email 3 – Missing student ID. The model correctly set `student_id` to `null`, which shows it understood that the ID was not present. However, it did not populate the `missing_information` array. This is likely because the prompt gave an example format at the end with an empty list, and the model followed the example rather than reasoning about which fields were actually missing. A fix would be to rewrite the instruction for `missing_information` more explicitly, or to remove the empty list from the output example.

Email 4 – Ambiguous intent. The model failed to detect the ambiguity in email 4. The student said they "registered last semester" but were "not sure if I should stay or withdraw." The model interpreted "registered" as the current intent and returned `register` with high confidence. This suggests the model did not apply the rule about setting intent to `unknown` when unclear. Small models often struggle with negation and uncertainty detection. One way to improve this would be to include an example in the prompt that shows an ambiguous case and the expected output.

Email 5 – Extra irrelevant information. The model handled this well. It ignored sentences about the weather and hiking and focused on the relevant parts of the email. This is a good result because filtering noise is important in real-world use.

6 Conclusion

This assignment shows that a small instruction-tuned model can reliably extract structured information from student emails and produce valid JSON output when given a well-defined prompt. The system achieved a 5/5 valid JSON rate and was fully correct on 3 out of 5 emails.

The main weaknesses were in handling the `missing_information` field and detecting ambiguous intent. Both issues point to limitations in how small models follow multi-step conditional instructions. These could be addressed by revising the prompt with more explicit instructions and a few-shot example for the edge cases.

Overall, the prompt contract approach is effective for this kind of information extraction task. Treating the prompt as a structured specification rather than a conversational message improved the reliability of the output format.

References

- [1] Qwen Team. *Qwen2.5 Technical Report*. arXiv preprint arXiv:2412.15115, 2025.
- [2] T. Wolf, L. Debut, V. Sanh, et al. *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020: System Demonstrations, pp. 38–45, 2020.

A Full JSON Outputs

Listing 2: Email 1 – Clear registration request

```
{
  "student_name": "John Smith",
  "student_id": "2045123",
  "email_intent": "register",
  "course_or_exam": "Advanced Topics in Neurocomputing",
  "confidence": "high",
  "missing_information": []
}
```

Listing 3: Email 2 – Clear deregistration request

```
{
  "student_name": "Maria Garcia",
  "student_id": "2045456",
  "email_intent": "deregister",
  "course_or_exam": "Advanced Topics in Neurocomputing",
  "confidence": "high",
  "missing_information": []
}
```

```
}
```

Listing 4: Email 3 – Missing student ID

```
{  
  "student_name": "Ahmed Hassan",  
  "student_id": null,  
  "email_intent": "register",  
  "course_or_exam": "Quantum Computing module",  
  "confidence": "high",  
  "missing_information": []  
}
```

Listing 5: Email 4 – Ambiguous intent

```
{  
  "student_name": "Lisa Chen",  
  "student_id": "2045789",  
  "email_intent": "register",  
  "course_or_exam": "Advanced Topics in Neurocomputing",  
  "confidence": "high",  
  "missing_information": []  
}
```

Listing 6: Email 5 – Extra irrelevant information

```
{  
  "student_name": "Tom Brown",  
  "student_id": "2045321",  
  "email_intent": "register",  
  "course_or_exam": "AI Agents",  
  "confidence": "high",  
  "missing_information": []  
}
```